

Ai Analysis: I'm sticking with Claude and Gemini as my Ai's of choice because those have worked the best for me in the past in terms of clean code. My first prompt for both was the rubric and the instructions for the assignment. I had to prompt Claude to generate the code because it gave me an overview of my assignment with the first prompt. I decided to use Claude for the final code because it generated more sophisticated code with edge case checks. I asked Claude to make improvements on the code and it suggested to sort emails with tied priority by earliest arrived first and preventing crashes by skipping invalid instructions. It also suggested these performance improvements as well as making it more maintainable

- **Time**

- Output uses `'\n'` instead of `std::endl`, so it no longer flushes on every line.
- Emails are moved into the heap rather than copied, and the vector is pre-reserved.
- I turned off stream syncing with `sync_with_stdio(false)`.
- Big-O is unchanged at $O(\log n)$ for insert and remove and $O(1)$ for peek.

- **Space**

- The sender is now a small enum instead of a string.
- The date is stored as one int instead of a string, plus an int for ordering.
- It's still $O(n)$ overall, with a smaller per-email footprint.

- **Maintainability**

- Ranks are a named enum instead of the magic numbers 1 to 5.
- `parseEmail`, `parseSender` and `parseDate` are separate from command dispatch.